Отчёт по лабораторной работе №24

по курсу «Фундаментальная информатика и информационные технологии».

Выполнил студент группы **М8О-114БВ-25 Сабирзянов Адель Рамилевич** № по списку 20.

Контакты: sabiadel386@gmail.com

Работа выполнена: «19» апреля 2026 г.

Преподаватель: каф.806 Сысоев Максим Алексеевич

Входной контроль знаний с оценкой: _____

Отчет сдан «21» апреля 2026 г.

Итоговая оценка: _____

Подпись преподавателя: _____

1. Тема

Динамические структуры данных. Бинарные деревья.

2. Цель работы

Изучение способов представления бинарных деревьев в памяти компьютера, освоение алгоритмов построения дерева из инфиксной записи выражения и реализация рекурсивных алгоритмов трансформации структуры дерева.

3. Задание (Вар. 20)

Реализовать программу для работы с деревом выражений, поддерживающую следующие операции:

Построение дерева на основе текстового математического выражения (алгоритм Дейкстры).

Визуализация дерева в текстовом виде (инфиксная запись) и в виде иерархической структуры.

Специфическая задача: Реализовать тождественные преобразования для упрощения дробей: (a / b) / c = a / (b * c) , a / (b / c) = (a / b) * c

Корректная очистка всей динамической памяти.

4. Оборудование

Процессор: 13th Gen Intel(R) Core(TM) i5-13420H @ 2.10 GHz, ОП 8 ГБ, SSD 1 ТБ

5. Программное обеспечение

Операционная система: Ubuntu 24.04 LTS (WSL), компилятор gcc версия 13.2.0, редактор VS Code.

6. Идея, метод, алгоритм решения задачи

Парсинг: Использование алгоритма «Сортировочной станции» с двумя стеками: один для операторов (char), второй для указателей на узлы дерева (Node*).

Хранение: Бинарное дерево, где каждый узел содержит массив символов для данных и два указателя на потомков.

Трансформация: Рекурсивный обход дерева. При обнаружении узла деления проверяется наличие аналогичного оператора в дочерних узлах. При совпадении паттерна выполняется перестройка указателей left и right и изменение данных в узлах через strcpy.

7. Сценарий выполнения работы

1. Определение структуры Node и вспомогательной структуры StackNode для реализации стека на связном списке.
2. Написание функций push и pop для управления стеком указателей.
3. Реализация: чтение строки, выделение токенов (чисел/переменных) и сборка иерархии с учетом приоритетов.
4. Написание рекурсивной функции transform для реализации логики варианта №20.
5. Добавление функций визуализации: printInfix (со скобками) и printTree (с отступами).

Текст программы:

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <ctype.h>


typedef struct Node {

    char data[16];

    struct Node *left;

    struct Node *right;

} Node;


Node* createNode(const char* value) {

    Node* newNode = (Node*)malloc(sizeof(Node));

    if (!newNode) return NULL;


    strncpy(newNode->data, value, 15);
```

```
    newNode->data[15] = '\0';

    newNode->left = NULL;
    newNode->right = NULL;
    return newNode;
}

void freeTree(Node* root) {
    if (root == NULL) return;
    freeTree(root->left);
    freeTree(root->right);
    free(root);
}

typedef struct StackNode {
    Node* treeNode;
    struct StackNode* next;
} StackNode;

void push(StackNode** top, Node* tNode) {
    StackNode* newNode = (StackNode*)malloc(sizeof(StackNode));
    if (!newNode) {
        return;
    }
    newNode->treeNode = tNode;
    newNode->next = *top;
    *top = newNode;
}
```

```
Node* pop(StackNode** top) {
    if (*top == NULL) {
        return NULL;
    }
    StackNode* temp = *top;
    Node* tNode = temp->treeNode;
    *top = (*top)->next;
    free(temp);
    return tNode;
}

int isEmpty(StackNode* top) {
    return top == NULL;
}

int getPriority(char op) {
    switch (op) {
        case '+':
        case '-':
            return 1;
        case '*':
        case '/':
            return 2;
        case '(':
        case ')':
            return 0;
        default:
            return -1;
    }
}
```

```
void buildSubTree(StackNode** nodes, char op) {
    char strOp[2] = {op, '\0'};
    Node* root = createNode(strOp);

    root->right = pop(nodes);
    root->left = pop(nodes);

    push(nodes, root);
}

Node* parseExpression(const char* expr) {
    StackNode* nodes = NULL;      // Стек для Node*
    char ops[100];
    int opsTop = -1;

    for (int i = 0; expr[i] != '\0'; i++) {
        if (expr[i] == ' ') continue;

        // число или буква
        if (isalnum(expr[i]) || expr[i] == '.') {
            char buffer[16] = {0};
            int k = 0;
            while (isalnum(expr[i]) || expr[i] == '.') {
                buffer[k++] = expr[i++];
            }
            i--;
            push(&nodes, createNode(buffer));
        }
        // открывающая скобка
```

```
        else if (expr[i] == '(') {
            ops[++opsTop] = '(';
        }
        // закрывающая скобка
        else if (expr[i] == ')') {
            while (opsTop >= 0 && ops[opsTop] != '(') {
                buildSubTree(&nodes, ops[opsTop--]);
            }
            opsTop--;
        }
        // оператор
        else {
            while (opsTop >= 0 && ops[opsTop] != '(' &&
                  getPriority(ops[opsTop]) >= getPriority(expr[i])) {
                buildSubTree(&nodes, ops[opsTop--]);
            }
            ops[++opsTop] = expr[i];
        }
    }

    while (opsTop >= 0) {
        buildSubTree(&nodes, ops[opsTop--]);
    }

    return pop(&nodes);
}

void printInfix(Node* root) {
    if (root == NULL) return;
```

```
    if (root->left != NULL || root->right != NULL) {
        printf("(");
    }

    printInfix(root->left);

    printf(" %s ", root->data);

    printInfix(root->right);

    if (root->left != NULL || root->right != NULL) {
        printf(")");
    }
}

void transform(Node* root) {
    if (root == NULL) return;

    transform(root->left);
    transform(root->right);

    // Проверяем, является ли текущий узел делением
    if (strcmp(root->data, "/") == 0) {

        //(a / b) / c  = a / (b * c)
        if (root->left != NULL && strcmp(root->left->data, "/") == 0) {
            Node* A = root->left->left;
            Node* B = root->left->right;
            Node* C = root->right;
            Node* leftSlash = root->left;
```

```
    root->left = A;

    strcpy(leftSlash->data, "*");
    leftSlash->left = B;
    leftSlash->right = C;

    root->right = leftSlash;

    transform(root);
}

// a / (b / c)  = (a / b) * c
else if (root->right != NULL && strcmp(root->right->data, "/") == 0) {
    Node* A = root->left;
    Node* B = root->right->left;
    Node* C = root->right->right;
    Node* rightSlash = root->right;

    strcpy(root->data, "*");

    strcpy(rightSlash->data, "/");
    rightSlash->left = A;
    rightSlash->right = B;

    root->left = rightSlash;
    root->right = C;

    transform(root);
}
```

```
    }
}

void printTree(Node* root, int level) {
    if (root == NULL) return;

    printTree(root->right, level + 1);

    for (int i = 0; i < level; i++) printf("    ");
    printf("%s\n", root->data);

    printTree(root->left, level + 1);
}

int main() {
    char input[100];
    printf("Input: ");
    fgets(input, 100, stdin);

    Node* root = parseExpression(input);

    printf("Start: ");
    printInfix(root);
    printf("\n");

    transform(root);

    printf("End: ");
    printInfix(root);
    printf("\n");
```

```
    printTree(root, 0);


    freeTree(root);
    return 0;
}
```

Пункты 1-7 отчета составляются строго до начала лабораторной работы.

Допущен к выполнению работы. Подпись преподавателя _____

8. Распечатка протокола

Input: (a+b)/(c*s)/(a*d-l)

Start: (((a + b) / (c * s)) / ((a * d) - l))

End: ((a + b) / ((c * s) * ((a * d) - l)))

Структура дерева:

```
        -
            l
        *
            d
            a
    *
        *
            s
            c
/
    +
        b
        a
```

9. Дневник отладки

№	Лаб. или дом.	Дата	Время	Событие	Действие по исправлению	Примечание

10. Замечания автора по существу работы

Использование стека на связном списке позволяет обрабатывать выражения произвольной длины без переполнения статического буфера. Рекурсивный подход к трансформации позволяет обрабатывать вложенные дроби любой глубины.

11. Выводы

В ходе работы были освоены принципы построения деревьев выражений. Реализован алгоритм тождественных преобразований для оптимизации структуры дерева. Закреплены навыки управления динамической памятью и работы с указателями в языке Си.

Подпись студента _____